



UNIVERSITY OF DHAKA

Department of Computer Science and Engineering

Faculty of Engineering and Technology (FoET)

Second Science Complex, Mokarram Bhaban Area, Dhaka 1000, Bangladesh

CSE 2205 — Microcontroller and Embedded System

STM32F446RE Nucleo-64 Assignment 01 Manual

2nd Year, 2nd Semester 2024

ARM Cortex-M4 | Bare-Metal Register Programming | HAL Driver Development

Course Code:	CSE 2205
Current [mmyyyy]:	Feb, 2026
Credits:	3.0 Units
Prerequisites:	CSE 2101 (DSA), CSE 2105 (Computer Architecture)
Board:	STM32F446RE Nucleo-64 (ARM Cortex-M4, 180 MHz)
IDE:	STM32CubeIDE 1.14+ with STM32CubeMX
Toolchain:	GNU Arm Embedded Toolchain (arm-none-eabi-gcc)

This assignment contains five Labs using Bare-Metal and STM32CubeIde - HAL (Hardware Abstraction Layer)

Lab 1	Lab 2	Lab 3	Lab 4	Lab 5
LED Blink GPIO	UART Polling USART2	Ext. Interrupt EXTI / NVIC	UART Interrupt Ring Buffer	UART DMA DMA1



Contents

Introduction & Lab Setup	2
1 LED Blink — GPIO Output Control	4
1.1 Background — GPIO Register Map	4
1.2 Lab 1A — No HAL Code	4
1.3 Lab 1B — HAL Code	5
2 UART Polling — Echo Server	7
2.1 Background — USART on STM32F4	7
2.2 Lab 2A — No HAL Code	7
2.3 Lab 2B — HAL Code	8
3 External Interrupt — Button Toggle (EXTI)	10
3.1 Background — STM32 Interrupt Architecture	10
3.2 Lab 3A — No HAL Code	10
3.3 Lab 3B — HAL Code	11
4 UART Receive Interrupt — Ring Buffer	13
4.1 Background — Ring Buffer Design	13
4.2 Lab 4A — No HAL Code	13
4.3 Lab 4B — HAL Code	14
5 UART DMA — Background Data Transfer	17
5.1 Background — DMA1 Controller	17
5.2 Lab 5A — No HAL Code	18
5.3 Lab 5B — HAL Code	19
A Debugging Techniques	21
A.1 GNU Debugger (GDB) in STM32CubeIDE	21
A.2 Common Fault Table	21
B Register Quick Reference	22
B.1 RCC — Clock Enable	22
B.2 GPIO — Bit-Field Template	22
B.3 NVIC — CMSIS Helpers	22
B.4 Useful Macros	22
C Assessment Rubric	23

Introduction & Lab Setup

This lab manual accompanies **CSE 2205 — Microcontroller and Embedded System** for 2nd year, 2nd semester students at the University of Dhaka. Five progressive laboratory sessions guide students from bare-metal, register-level programming to HAL-abstracted driver development on the **STM32F446RE Nucleo-64** development board.

Each lab is presented in two parallel versions:

- **No-HAL (Bare-Metal):** Direct access to hardware registers using CMSIS structs. Requires reading the RM0390 reference manual and datasheet. Develops deep hardware understanding.
- **HAL:** ST's Hardware Abstraction Layer with CubeMX code generation. Portable, faster to develop, and used widely in industry for rapid prototyping.

Hardware Requirements

- STM32F446RE Nucleo-64 development board (ARM Cortex-M4, 180 MHz max, 512 KB Flash, 128 KB SRAM)
- Micro-USB cable (powers board and connects to ST-LINK/V2-1 on-board debugger)
- Host PC: Windows 10/11 or Ubuntu 20.04+
- Logic analyser or oscilloscope (for post-lab waveform analysis)

Software Requirements

- **STM32CubeIDE 1.14+** — <https://www.st.com/en/development-tools/stm32cubeide.html>
- STM32CubeMX (bundled inside CubeIDE) — for HAL peripheral configuration and code generation
- Serial terminal: **PuTTY** (Windows) or **screen** / **minicom** (Linux) — 115200 baud, 8N1
- Reference: *RM0390 Rev 6* — STM32F446xx Reference Manual (STMicroelectronics)

Board Quick-Reference

Signal	MCU Pin	Notes
LD2 (User LED)	PA5	Active HIGH; push-pull output
B1 (User Button)	PC13	Active LOW; on-board pull-up resistor
USART2 TX (VCP)	PA2	Wired to ST-LINK virtual COM port
USART2 RX (VCP)	PA3	Wired to ST-LINK virtual COM port
ST-LINK USB	CN1	Programming, debug (SWD), and VCP
VDD	3.3 V	On-board LDO from USB or external supply

Table 1: Nucleo-F446RE signal assignments used in these labs

Creating a Project in STM32CubeIDE

For No-HAL projects: *File* → *New* → *STM32 Project* → *Board Selector* → “*NUCLEO-F446RE*” → *Finish*. Delete HAL initialisation calls from `main.c`. Retain only the startup file (`startup_stm32f446retx.s`) and linker script. Add `stm32f4xx.h` and the CMSIS device header to your include path.

For HAL projects: Use the same board selector, then open the generated `.ioc` file in the CubeMX perspective to configure peripherals graphically. Click *Project* → *Generate Code*. Write application logic *only inside* `/* USER CODE BEGIN */ ... /* USER CODE END */` markers so re-generation does not overwrite your work.

!! Critical Rule

Never modify generated code outside USER CODE sections. STM32CubeMX will silently overwrite it on the next code-generation run.

No-HAL vs. HAL: Conceptual Comparison

Aspect	No-HAL (Bare-Metal)	HAL
Abstraction	Direct register access via CMSIS struct	Portable HAL API functions
Code size	Minimal — only what you write	Larger due to abstraction layers
Portability	Tied to STM32F4 family	Portable across STM32 families
Debug ease	Inspect SFRs directly in debugger	Use HAL status return codes
Development speed	Slower; requires datasheet study	Faster with CubeMX code generation
Industry use	Safety-critical, size-constrained	Rapid prototyping, IoT products

Table 2: No-HAL vs. HAL: trade-offs at a glance

Lab 1 LED Blink — GPIO Output Control

Lab 1A (No HAL) & Lab 1B (HAL)

CO1, CO2

★ Introductory

[OK] Learning Objectives

- Enable a GPIO peripheral clock via the **RCC->AHB1ENR** register
- Configure a GPIO pin as push-pull output using **MODER**, **OTYPER**, **OSPEEDR**, **PUPDR**
- Toggle an output atomically using the Bit Set/Reset Register (**BSRR**)
- Implement a blocking software delay without a timer peripheral
- Compare and contrast No-HAL and HAL approaches for identical functionality

1.1 Background — GPIO Register Map

Each GPIO port (A–H on STM32F446) is mapped to a 0x400-byte region on the AHB1 bus starting at 0x40020000 (GPIOA). Key configuration registers:

Register	Bits/Pin	Description
MODER	2	00=Input, 01=Output, 10=Alternate Function, 11=Analog
OTYPER	1	0=Push-Pull, 1=Open-Drain
OSPEEDR	2	00=Low, 01=Medium, 10=High, 11=Very High
PUPDR	2	00=None, 01=Pull-Up, 10=Pull-Down
ODR	1	Output Data Register — not atomic (read-modify-write)
BSRR	—	Bits[15:0]=Set, Bits[31:16]=Reset — atomic, write-only

Table 1.1: Key GPIO configuration registers for STM32F446

[i] Key Insight — BSRR vs. ODR

Always use **BSRR** instead of **ODR** for setting/clearing individual bits in ISRs. **ODR** requires a non-atomic read-modify-write that can cause race conditions if an interrupt fires between the read and write. **BSRR** writes a single 32-bit word that atomically sets or clears pins with no intermediate state.

1.2 Lab 1A — No HAL Code

```
1 /* Lab 1A - LED Blink (No HAL) on STM32F446RE Nucleo
2  * PA5 = LD2 (User LED) on Nucleo-F446RE
3  * Clock source: HSI 16 MHz (default after reset)
4  */
5 #include "stm32f4xx.h"
```

```

6
7 void delay_ms(uint32_t ms) {
8     /* Software delay: ~16 MHz HSI -> ~4,000 NOP iterations per ms */
9     for (uint32_t i = 0; i < ms * 4000; i++) {
10         __NOP();
11     }
12 }
13
14 int main(void) {
15     /* 1. Enable GPIOA clock on AHB1 bus */
16     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
17     __NOP(); __NOP(); /* brief settle delay (errata recommendation) */
18
19     /* 2. MODER: set PA5 as General Purpose Output (bits [11:10] = 01) */
20     GPIOA->MODER &= ~(3UL << (5 * 2)); /* clear bits */
21     GPIOA->MODER |= (1UL << (5 * 2)); /* output mode */
22
23     /* 3. OTYPER: Push-Pull (bit5 = 0, reset default) */
24     GPIOA->OTYPER &= ~(1UL << 5);
25
26     /* 4. OSPEEDR: Low speed (bits [11:10] = 00, reset default) */
27     GPIOA->OSPEEDR &= ~(3UL << (5 * 2));
28
29     /* 5. PUPDR: No pull (bits [11:10] = 00, reset default) */
30     GPIOA->PUPDR &= ~(3UL << (5 * 2));
31
32     while (1) {
33         GPIOA->BSRR = (1UL << 5); /* Set PA5 -> LED ON */
34         delay_ms(500);
35         GPIOA->BSRR = (1UL << (5 + 16)); /* Reset PA5 -> LED OFF */
36         delay_ms(500);
37     }
38 }

```

Listing 1.1: Lab 1A: LED blink via direct register access (No HAL)

1.3 Lab 1B — HAL Code

```

1  /* Lab 1B - LED Blink using HAL on STM32F446RE Nucleo
2   * CubeMX config: PA5 (LD2_Pin) -> GPIO Output, Push-Pull, Low speed
3   */
4  #include "main.h"
5
6  int main(void) {
7      HAL_Init(); /* Init HAL, configure SysTick (1 ms timebase) */
8      SystemClock_Config(); /* Configure system clock (CubeMX generated) */
9      MX_GPIO_Init(); /* Init GPIO (CubeMX generated) */
10
11     while (1) {
12         HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin); /* atomic toggle */
13         HAL_Delay(500); /* 500 ms SysTick-based delay */
14     }
15 }
16
17 /* CubeMX-generated GPIO init -- shown for comparison with Lab 1A */
18 void MX_GPIO_Init(void) {
19     GPIO_InitTypeDef GPIO_InitStruct = {0};
20     __HAL_RCC_GPIOA_CLK_ENABLE();
21
22     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
23
24     GPIO_InitStruct.Pin = LD2_Pin; /* PA5 */
25     GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;

```

```
26     GPIO_InitStruct.Pull    = GPIO_NOPULL;
27     GPIO_InitStruct.Speed   = GPIO_SPEED_FREQ_LOW;
28     HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
29 }
```

Listing 1.2: Lab 1B: LED blink using HAL (CubeMX generated init shown)

[Q] Post-Lab Questions

- Q1.** Why must **RCC->AHB1ENR** be set before accessing any GPIOA registers? What happens (at the silicon level) if you skip this step?
- Q2.** Calculate the bit positions for **MODER[PA8]**. Write the C expression to set PA8 as alternate-function mode without affecting any other pin. Show a register bit-field diagram.
- Q3.** What is the difference between **ODR** and **BSRR** for controlling GPIO output? Why is **BSRR** preferred in interrupt-safe code? Give a concrete race-condition scenario with **ODR**.
- Q4.** Measure the actual LED blink period using an oscilloscope or logic analyser. Calculate the overhead introduced by the delay loop and explain any discrepancy from the expected 1-second period.
- Q5.** Modify the code to produce a 25% duty cycle (ON 250ms, OFF 750ms). Show both No-HAL and HAL versions. Then generalise to a function `led_blink(uint32_t on_ms, uint32_t off_ms)`.

clock

[17:16]

ODR:atomic
BSRR: non-atomic

Lab 2 UART Polling — Echo Server

Lab 2A (No HAL) & Lab 2B (HAL)

CO1, CO2

★★ Intermediate

[OK] Learning Objectives

- Configure USART2 on APB1 bus for 115200 baud, 8N1 operation
- Calculate and write the Baud Rate Register (BRR) from first principles
- Configure GPIO alternate function AF7 for UART TX/RX pins (PA2, PA3)
- Implement polling-based character transmission and reception using SR flags
- Build an interactive echo server over the ST-LINK virtual COM port

2.1 Background — USART on STM32F4

USART2 sits on the APB1 bus (max 45 MHz on STM32F446). The baud rate register value is:

$$\text{BRR} = \frac{f_{\text{CK}}}{\text{baud rate}} \quad (\text{OVER8} = 0, 16\times \text{oversampling}) \quad (2.1)$$

The 16-bit BRR field packs the mantissa into bits [15:4] and the 4-bit fraction (sixteenths) into bits [3:0].

[=] Baud Rate Calculation Example

$$f_{\text{APB1}} = 45 \text{ MHz}, \quad \text{baud} = 115200$$

$$\text{BRR} = \frac{45,000,000}{115,200} = 390.625$$

$$\begin{aligned} \text{Mantissa} &= 390 = 0x186; & \text{Fraction} &= 0.625 \times 16 = 10 = 0xA \\ \Rightarrow \text{BRR} &= (390 \ll 4) \mid 10 = 0x186A \end{aligned}$$

Key **SR** (Status Register) flags:

- **TXE** — Transmit data register empty; safe to write next byte
- **TC** — Transmission complete; all bits shifted out on the line
- **RXNE** — Receive data register not empty; data ready to read
- **ORE** — Overrun error; new byte arrived before previous one was read

Reading **USART->DR** clears **RXNE**; writing **DR** loads the transmit shift register.

2.2 Lab 2A — No HAL Code

```
1  /* Lab 2A - USART2 Polling TX/RX (No HAL) on STM32F446RE
2  * USART2: PA2=TX, PA3=RX | APB1 Bus | Baud 115200, 8N1
3  * APB1 clock assumed = 45 MHz (configure PLL accordingly)
4  */
```

```

5  #include "stm32f4xx.h"
6
7  #define APB1_CLK    45000000UL
8  #define BAUD_RATE   115200UL
9
10 void USART2_Init(void) {
11     /* 1. Enable clocks */
12     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
13     RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
14     __NOP(); __NOP();
15
16     /* 2. PA2, PA3 -> Alternate Function 7 (USART2) */
17     GPIOA->MODER  &= ~(3UL << 4) | (3UL << 6));
18     GPIOA->MODER  |= ((2UL << 4) | (2UL << 6)); /* AF mode */
19     GPIOA->AFR[0] &= ~(0xFUL << 8) | (0xFUL << 12));
20     GPIOA->AFR[0] |= ((7UL << 8) | (7UL << 12)); /* AF7 */
21     GPIOA->OTYPER &= ~(1UL << 2) | (1UL << 3)); /* Push-Pull */
22     GPIOA->OSPEEDR |= ((3UL << 4) | (3UL << 6)); /* Very High */
23
24     /* 3. Baud rate: BRR = 45,000,000 / 115,200 = 390.625
25      *      Mantissa = 390, Fraction = 0.625 * 16 = 10 */
26     USART2->BRR = (390 << 4) | 10;
27
28     /* 4. Enable USART, transmitter, receiver */
29     USART2->CR1 = USART_CR1_UE | USART_CR1_TE | USART_CR1_RE;
30 }
31
32 void USART2_SendChar(char c) {
33     while (!(USART2->SR & USART_SR_TXE)); /* wait TX buffer empty */
34     USART2->DR = (uint8_t)c;
35 }
36
37 void USART2_SendString(const char *str) {
38     while (*str) USART2_SendChar(*str++);
39 }
40
41 char USART2_RecvChar(void) {
42     while (!(USART2->SR & USART_SR_RXNE)); /* wait data ready */
43     return (char)USART2->DR; /* reading DR clears RXNE */
44 }
45
46 int main(void) {
47     USART2_Init();
48     USART2_SendString("STM32F446RE UART Polling Demo\r\n");
49     USART2_SendString("Type a character -- it will be echoed:\r\n");
50
51     while (1) {
52         char c = USART2_RecvChar(); /* blocking receive */
53         USART2_SendChar(c); /* echo back */
54         if (c == '\r') USART2_SendChar('\n');
55     }
56 }

```

Listing 2.1: Lab 2A: UART polling echo server (No HAL)

2.3 Lab 2B — HAL Code

```

1  /* Lab 2B - UART2 Polling TX/RX using HAL on STM32F446RE
2   * CubeMX config: USART2, 115200, 8N1, PA2/PA3, No HW flow ctrl
3   */
4  #include "main.h"
5  #include <string.h>
6

```

```

7  UART_HandleTypeDef huart2;
8
9  int main(void) {
10     HAL_Init();
11     SystemClock_Config();
12     MX_USART2_UART_Init();    /* CubeMX generated */
13
14     const char *welcome = "STM32 HAL UART Polling Demo\r\nEcho server:\r\n";
15     HAL_UART_Transmit(&huart2, (uint8_t*)welcome,
16                       strlen(welcome), HAL_MAX_DELAY);
17
18     uint8_t rxByte;
19     while (1) {
20         /* HAL_UART_Receive: blocks until 1 byte received or timeout */
21         if (HAL_UART_Receive(&huart2, &rxByte, 1, HAL_MAX_DELAY) == HAL_OK) {
22             HAL_UART_Transmit(&huart2, &rxByte, 1, HAL_MAX_DELAY);
23             if (rxByte == '\r') {
24                 uint8_t nl = '\n';
25                 HAL_UART_Transmit(&huart2, &nl, 1, HAL_MAX_DELAY);
26             }
27         }
28     }
29 }
30
31 /* CubeMX-generated UART init (shown for reference) */
32 void MX_USART2_UART_Init(void) {
33     huart2.Instance      = USART2;
34     huart2.Init.BaudRate  = 115200;
35     huart2.Init.WordLength = UART_WORDLENGTH_8B;
36     huart2.Init.StopBits  = UART_STOPBITS_1;
37     huart2.Init.Parity     = UART_PARITY_NONE;
38     huart2.Init.Mode       = UART_MODE_TX_RX;
39     huart2.Init.HwFlowCtl  = UART_HWCONTROL_NONE;
40     huart2.Init.OverSampling = UART_OVERSAMPLING_16;
41     HAL_UART_Init(&huart2);
42 }

```

Listing 2.2: Lab 2B: UART polling echo server using HAL

[Q] Post-Lab Questions

- Q1.** Recalculate BRR for 9600 baud with $f_{APB1} = 45\text{ MHz}$. What are the mantissa and fraction values? What is the resulting actual baud rate and the percentage error?
- Q2.** What is the purpose of the **OVER8** bit in **CR1**? How does switching it to 1 affect baud rate accuracy and noise tolerance? Show the BRR formula change.
- Q3.** Polling continuously tests **TXE/RXNE** flags. Describe two scenarios where polling is acceptable and two where it wastes significant CPU cycles. What is the CPU utilisation during a polling receive at 115200 baud?
- Q4.** Trace through what happens byte-by-byte when **HAL_UART_Transmit()** sends the string "Hello". At what point does the function return? Is the string fully shifted out on the wire at that moment?
- Q5.** Add even parity to the No-HAL UART init. Which register bits control parity? Show the updated **CR1** write and draw the modified 9-bit frame format.

PCE = 1 ----> 8E1

Q2. 16 samples per bit → better majority voting
 → noise rejection
 8 samples → faster but less robust

Lab 3 External Interrupt — Button Toggle (EXTI)

Lab 3A (No HAL) & Lab 3B (HAL)

CO2 ★★ Intermediate

[OK] Learning Objectives

- Understand the STM32 interrupt system: NVIC, EXTI lines, SYSCFG mux
- Configure SYSCFG_EXTICRx to route a GPIO port to an EXTI line
- Set edge detection (falling/rising/both) and unmask the interrupt line
- Write an ISR, clear the pending flag correctly, and perform a LED toggle
- Understand NVIC preemption vs. sub-priority; set and reason about priorities

3.1 Background — STM32 Interrupt Architecture

The ARM Cortex-M4 NVIC manages up to 240 external interrupts plus 16 core exceptions. STM32F4 uses a two-level priority scheme:

- **Preemption priority** — determines whether this ISR can interrupt another running ISR
- **Sub-priority** — tie-breaking when two pending IRQs have the same preemption level

The EXTI controller provides 23 interrupt/event lines. Lines 0–15 map to GPIO pins (one line per pin number, across all ports). SYSCFG_EXTICRx selects which port (A–H) drives each line — because all ports share the same 16 line numbers.

the CPU only knows: “An interrupt occurred somewhere in lines 10–15”
It does NOT know which one.

[!] EXTI15_10 Shared Vector

Lines 10–15 all share a single ISR vector: EXTI15_10_IRQHandler. Inside the ISR you *must* check EXTI->PR to identify which line fired. Failure to clear the PR bit (by writing 1) causes the ISR to re-enter immediately on return — an infinite interrupt storm.

EXTI uses write-1-to-clear (W1C) semantics

3.2 Lab 3A — No HAL Code

```
1  /* Lab 3A - External Interrupt on PC13 (User Button) - No HAL
2  * PC13 = B1 (User Button, active LOW)
3  * Toggles PA5 (LD2) on each falling edge
4  */
5  #include "stm32f4xx.h"
6
7  void EXTI15_10_IRQHandler(void) {
8      if (EXTI->PR & (1UL << 13)) {          /* Is line 13 pending? */
9          EXTI->PR = (1UL << 13);             /* Clear pending (w1c) */
10         GPIOA->ODR ^= (1UL << 5);             /* Toggle LED */
11     }
12 }
13
14 int main(void) {
15     /* 1. Enable clocks */
16     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN | RCC_AHB1ENR_GPIOCEN;
17     RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;    /* SYSCFG for EXTI mux */
18 }
```

EXTI line 0 → pin number 0
...EXTI line 15 → pin number 15

PA0, PB0, PC0, ... all compete for EXTI line 0
PA13, PB13, PC13 → all map to EXTI line 13

```

18  __NOP(); __NOP();
19
20  /* 2. PA5 as push-pull output (LED) */
21  GPIOA->MODER &= ~(3UL << 10);
22  GPIOA->MODER |= (1UL << 10);
23
24  /* 3. PC13 as input (reset-default state is already input) */
25  GPIOC->MODER &= ~(3UL << 26); /* explicitly clear to input */
26
27  /* 4. SYSCFG EXTICR4: route Port C to EXTI line 13
28   *   EXTICR[3] controls lines 15:12; EXTI13 -> bits [7:4] = 0010 */
29  SYSCFG->EXTICR[3] &= ~(0xFUL << 4);
30  SYSCFG->EXTICR[3] |= (0x2UL << 4); /* 0x2 = Port C */
31
32  /* 5. EXTI line 13: falling-edge trigger, unmask */
33  EXTI->IMR |= (1UL << 13); /* unmask interrupt */
34  EXTI->RTSR &= ~(1UL << 13); /* no rising edge */
35  EXTI->FTSR |= (1UL << 13); /* falling edge only */
36
37  /* 6. Enable in NVIC (priority 1) */
38  NVIC_SetPriority(EXTI15_10_IRQn, 1);
39  NVIC_EnableIRQ(EXTI15_10_IRQn);
40
41  while (1) {
42      __WFI(); /* Wait For Interrupt -- sleep until next event */
43  }
44 }

```

Listing 3.1: Lab 3A: Button-controlled LED via EXTI (No HAL)

3.3 Lab 3B — HAL Code

```

1  /* Lab 3B - External Interrupt on PC13 using HAL
2   * CubeMX: PC13 -> GPIO_EXTI13, falling edge, NVIC preemption priority 0
3   */
4  #include "main.h"
5
6  /* HAL EXTI callback -- called from stm32f4xx_hal_gpio.c ISR stub */
7  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
8      if (GPIO_Pin == B1_Pin) { /* B1_Pin = GPIO_PIN_13 */
9          HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
10     }
11 }
12
13 int main(void) {
14     HAL_Init();
15     SystemClock_Config();
16     MX_GPIO_Init(); /* CubeMX: PC13 as EXTI input, PA5 as output */
17
18     while (1) {
19         __WFI(); /* Main loop does nothing; ISR handles all work */
20     }
21 }
22
23 /* CubeMX-generated MX_GPIO_Init (shown for transparency) */
24 void MX_GPIO_Init(void) {
25     GPIO_InitTypeDef GPIO_InitStruct = {0};
26     __HAL_RCC_GPIOA_CLK_ENABLE();
27     __HAL_RCC_GPIOC_CLK_ENABLE();
28
29     /* LED output */
30     HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
31     GPIO_InitStruct.Pin = LD2_Pin;

```

```

32     GPIO_InitStruct.Mode   = GPIO_MODE_OUTPUT_PP;
33     GPIO_InitStruct.Pull   = GPIO_NOPULL;
34     GPIO_InitStruct.Speed  = GPIO_SPEED_FREQ_LOW;
35     HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
36
37     /* Button: falling-edge EXTI */
38     GPIO_InitStruct.Pin     = B1_Pin;
39     GPIO_InitStruct.Mode    = GPIO_MODE_IT_FALLING;
40     GPIO_InitStruct.Pull    = GPIO_NOPULL;
41     HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);
42
43     HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
44     HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
45 }

```

Listing 3.2: Lab 3B: Button-controlled LED via EXTI using HAL

[Q] Post-Lab Questions

- Q1.** Explain the **SYSCFG** clock enable step. What happens at the bus level if you write to **SYSCFG->EXTICR** before enabling the SYSCFG clock?
- Q2.** The **B1 button** on Nucleo-F446RE has **mechanical contact bounce**. Using an oscilloscope, measure the bounce duration. Then implement software debouncing in the ISR using a SysTick-based timestamp (**HAL_GetTick()**). Show your modified ISR code.
- Q3.** What is the difference between falling-edge and both-edge detection (**FTSR** vs. **FTSR + RTSR**)? Give an application scenario where you would choose each.
- Q4.** The EXTI ISR is at NVIC priority 1. If a UART ISR is running at priority 0 and the button is pressed, will the EXTI ISR preempt the UART ISR? Justify with reference to the NVIC priority scheme.
- Q5.** Implement a **state machine** inside the callback so that three consecutive button presses cycle through three LED blink patterns: 1 Hz, 2 Hz, and OFF. Use a **volatile** state variable and a **TIM6 basic timer** for the blink timing.

Q4. NVIC priority and preemption

UART ISR priority = 0 (higher priority)

EXTI ISR priority = 1 (lower priority)

Rule (Cortex-M): Lower numerical value = higher priority

Scenario

UART ISR is executing

Button press occurs → EXTI interrupt pending

Will EXTI preempt UART? No. Because: EXTI (priority 1) is lower priority

It must wait until UART ISR completes

Reverse case

If EXTI = 0 and UART = 1: EXTI would preempt UART

Conclusion

NVIC enforces strict priority-based preemption

No round-robin or fairness

Lab 4 UART Receive Interrupt — Ring Buffer

Lab 4A (No HAL) & Lab 4B (HAL)

CO2 ★★★ Advanced

[OK] Learning Objectives

- Enable the UART RXNE interrupt (`CR1_RXNEIE`) and write a correct ISR
- Implement a **lock-free ring buffer** for **single-producer**, **single-consumer** data transfer between ISR and main loop
- Handle UART overrun errors (`ORE`) gracefully inside the ISR
- Understand the difference between **polling** and **interrupt-driven I/O** in terms of **CPU utilisation** and **response latency**

4.1 Background — Ring Buffer Design

A circular (ring) buffer **decouples** the **producer** (ISR writing bytes at interrupt time) from the **consumer** (main loop reading bytes) **without** needing a mutex, *provided* there is exactly one producer and one consumer, and the head/tail indices are declared **volatile**.

- Buffer **empty**: `head == tail`
- Buffer **full**: `(head + 1) % SIZE == tail`
- **ISR increments head** after writing; **main loop increments tail** after reading

lock-free single-producer / single-consumer (SPSC) queue

[!] Volatile Correctness

Declare `head`, `tail`, and the buffer array as **volatile** to prevent the compiler from caching them in CPU registers and **optimising away** the ISR-visible updates. Without **volatile**, the main loop may never observe changes made in the ISR.

With volatile: Compiler is forced to reload from memory every time as no caching in register and thus visible updates

4.2 Lab 4A — No HAL Code

```
1  /* Lab 4A - UART2 Receive Interrupt + Ring Buffer (No HAL)
2   * Receives bytes in ISR into a ring buffer.
3   * Main loop detects complete lines and echoes them.
4   */
5  #include "stm32f4xx.h"
6  #include <string.h>
7  #include <stdio.h>
8
9  #define BUF_SIZE 128
10 volatile char    ring[BUF_SIZE];
11 volatile uint8_t head = 0, tail = 0;
12 volatile uint8_t lineReady = 0;
13
14 /* USART2 ISR */
15 void USART2_IRQHandler(void) {
16     if (USART2->SR & USART_SR_RXNE) {
17         char c = (char)USART2->DR;    /* reading DR clears RXNE    */
18     }
```

```

18     ring[head] = c;
19     head = (head + 1) % BUF_SIZE;
20     if (c == '\r') lineReady = 1;
21 }
22 if (USART2->SR & USART_SR_ORE) {
23     (void)USART2->DR;          /* reading DR also clears ORE */
24 }
25 }
26
27 void USART2_SendString(const char *s) {
28     while (*s) {
29         while (!(USART2->SR & USART_SR_TXE));
30         USART2->DR = (uint8_t)(*s++);
31     }
32 }
33
34 int main(void) {
35     /* GPIOA + USART2 clock and GPIO AF config (same as Lab 2A) */
36     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
37     RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
38     __NOP(); __NOP();
39     GPIOA->MODER  &= ~((3UL<<4)|(3UL<<6));
40     GPIOA->MODER  |= ((2UL<<4)|(2UL<<6));
41     GPIOA->AFR[0] &= ~((0xFUL<<8)|(0xFUL<<12));
42     GPIOA->AFR[0] |= ((7UL<<8)|(7UL<<12));
43
44     USART2->BRR = (390 << 4) | 10;    /* 115200 @ 45 MHz */
45     USART2->CR1 = USART_CR1_UE | USART_CR1_TE
46                 | USART_CR1_RE | USART_CR1_RXNEIE; /* RXNE IRQ en */
47
48     NVIC_SetPriority(USART2_IRQn, 1);
49     NVIC_EnableIRQ(USART2_IRQn);
50
51     USART2_SendString("UART Interrupt Demo - type and press Enter:\r\n");
52
53     while (1) {
54         if (lineReady) {
55             lineReady = 0;
56             /* Drain ring buffer into a local line buffer */
57             char line[BUF_SIZE];
58             uint8_t i = 0;
59             while (tail != head) {
60                 line[i++] = ring[tail];
61                 tail = (tail + 1) % BUF_SIZE;
62             }
63             line[i] = '\0';
64             char resp[BUF_SIZE + 16];
65             snprintf(resp, sizeof(resp), "Echo: %s\r\n", line);
66             USART2_SendString(resp);
67         }
68     }
69 }

```

Listing 4.1: Lab 4A: UART RX interrupt with ring buffer (No HAL)

4.3 Lab 4B — HAL Code

```

1  /* Lab 4B - UART2 Receive Interrupt using HAL
2   * HAL_UART_Receive_IT: non-blocking, fires callback after N bytes
3   */
4  #include "main.h"
5  #include <string.h>
6  #include <stdio.h>

```

```

7
8  UART_HandleTypeDef huart2;
9  uint8_t  rxByte;
10 char      lineBuf[128];
11 uint8_t   lineIdx = 0;
12
13 /* Called by HAL after each byte received (re-arms itself) */
14 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
15     if (huart->Instance == USART2) {
16         char c = (char)rxByte;
17         if (c == '\r' || lineIdx >= 127) {
18             lineBuf[lineIdx] = '\0';
19             char resp[160];
20             int len = snprintf(resp, sizeof(resp),
21                               "Echo: %s\r\n", lineBuf);
22             /* Use blocking TX from callback (safe if TX priority <= RX) */
23             HAL_UART_Transmit(&huart2, (uint8_t*)resp, len, HAL_MAX_DELAY);
24             lineIdx = 0;
25         } else {
26             lineBuf[lineIdx++] = c;
27         }
28         /* Re-arm for next byte */
29         HAL_UART_Receive_IT(&huart2, &rxByte, 1);
30     }
31 }
32
33 int main(void) {
34     HAL_Init();
35     SystemClock_Config();
36     MX_USART2_UART_Init();
37
38     const char *msg = "HAL UART Interrupt Demo\r\n";
39     HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), HAL_MAX_DELAY);
40
41     /* Arm the first receive -- callback will re-arm on each completion */
42     HAL_UART_Receive_IT(&huart2, &rxByte, 1);
43
44     while (1) {
45         /* Main loop is free; enter sleep to save power */
46         HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
47     }
48 }

```

Listing 4.2: Lab 4B: UART RX interrupt using HAL_UART_Receive_IT

[Q] Post-Lab Questions

- Q1.** Prove that the single-producer, single-consumer ring buffer above is thread-safe without a mutex, given that 32-bit aligned integer reads/writes are atomic on Cortex-M4. Identify the exact memory-ordering assumptions required.
- Q2.** What is an overrun error (ORE)? At what combination of baud rate and main-loop latency does it become a risk with a 128-byte buffer? Implement an overrun counter and print it over UART every second.
- Q3.** Calculate the **maximum sustained throughput** (bytes/second) this interrupt-driven design can support before the ring buffer overflows, given a 128-byte buffer and a 1 ms main-loop iteration that drains the buffer completely each pass.
- Q4.** HAL_UART_Receive_IT() is re-armed inside the callback. What happens if the callback execution time exceeds 1 byte-time at 115200 baud? Measure the callback execution time with a GPIO toggle and oscilloscope, and propose a mitigation. ORE

Q5. Extend the design to use TX interrupts (TXEIE) instead of blocking `HAL_UART_Transmit`. Implement a TX ring buffer drained by the UART TXE ISR. Show the full No-HAL implementation.

A mutex (short for mutual exclusion) is a synchronization primitive used to ensure that only one execution context at a time can access a shared resource.

Lab 5 UART DMA — Background Data Transfer

Lab 5A (No HAL) & Lab 5B (HAL)

CO2, CO3

★★★ Advanced

[OK] Learning Objectives

- Understand the DMA1 controller architecture: streams, channels, direction, FIFO, burst
- Configure a DMA stream for memory-to-peripheral (UART TX) transfer
- Configure a DMA stream for peripheral-to-memory (UART RX) circular mode
- Link DMA streams to USART2 using the channel-selection bits (`SxCR_CHSEL`)
- Handle the DMA Transfer Complete interrupt for TX; understand circular RX background capture
- Quantify the CPU utilisation improvement of DMA over polling and interrupt-per-byte methods

5.1 Background — DMA1 Controller

STM32F446 includes two DMA controllers (DMA1 and DMA2), each with 8 streams. Each stream connects to one of 8 channels (hardware peripheral request lines). Correct stream/channel mapping is taken from **RM0390 Table 28** (DMA1):

Peripheral	Direction	Stream	Channel
USART2 TX	Mem→Periph	DMA1 Stream 6	Channel 4
USART2 RX	Periph→Mem	DMA1 Stream 5	Channel 4

Table 5.1: DMA1 stream/channel mapping for USART2 (from RM0390 Table 28)

Key **DMA stream registers**:

- **SxCR** — Control: channel select, direction, increment, interrupt enables, enable bit
- **SxNDTR** — Number of data items remaining (decremented by hardware each transfer)
- **SxMOAR** — Memory address (source for TX, destination for RX)
- **SxPAR** — Peripheral address (always `&USART2->DR`)
- **HISR/LISR** — Status flags (TC, HT, TE, DME, FE per stream)
- **HIFCR/LIFCR** — Clear flags (write 1 to clear)

[!] Channel Mismatch = Silent Failure

Selecting the wrong DMA channel causes silent failure: DMA appears enabled but the peripheral request never triggers a transfer. Always verify stream and channel from the RM0390 DMA request-mapping table before debugging further.

5.2 Lab 5A — No HAL Code

```

1  /* Lab 5A - UART2 DMA TX (Stream6 Ch4) + Circular RX (Stream5 Ch4)
2  * No HAL -- direct DMA register configuration
3  */
4  #include "stm32f4xx.h"
5  #include <string.h>
6
7  #define RX_BUF_SIZE  64
8  static char          txMsg[] = "STM32 No-HAL DMA UART Transmit Demo\r\n";
9  static volatile char rxBuf[RX_BUF_SIZE];
10
11 void DMA1_Init(void) {
12     RCC->AHB1ENR |= RCC_AHB1ENR_DMA1EN;
13     __NOP(); __NOP();
14
15     /* --- TX: DMA1 Stream6 Channel4 (Memory -> Peripheral) --- */
16     DMA1_Stream6->CR = 0; /* disable first */
17     while (DMA1_Stream6->CR & DMA_SxCR_EN); /* wait disabled */
18     DMA1_Stream6->PAR = (uint32_t)&USART2->DR;
19     DMA1_Stream6->CR = (4UL << DMA_SxCR_CHSEL_Pos) /* Channel 4 */
20                     | DMA_SxCR_DIR_0 /* Mem -> Periph */
21                     | DMA_SxCR_MINC /* Memory increment */
22                     | DMA_SxCR_TCIE; /* TC interrupt */
23
24     /* --- RX: DMA1 Stream5 Channel4 (Peripheral -> Memory, Circular) --- */
25     DMA1_Stream5->CR = 0;
26     while (DMA1_Stream5->CR & DMA_SxCR_EN);
27     DMA1_Stream5->PAR = (uint32_t)&USART2->DR;
28     DMA1_Stream5->MOAR = (uint32_t)rxBuf;
29     DMA1_Stream5->NDTR = RX_BUF_SIZE;
30     DMA1_Stream5->CR = (4UL << DMA_SxCR_CHSEL_Pos)
31                     | DMA_SxCR_CIRC /* Circular mode */
32                     | DMA_SxCR_MINC;
33     DMA1_Stream5->CR |= DMA_SxCR_EN; /* Enable RX DMA */
34
35     NVIC_SetPriority(DMA1_Stream6_IRQn, 1);
36     NVIC_EnableIRQ(DMA1_Stream6_IRQn);
37 }
38
39 /* TX Transfer Complete ISR */
40 void DMA1_Stream6_IRQHandler(void) {
41     if (DMA1->HISR & DMA_HISR_TCIF6) {
42         DMA1->HIFCR = DMA_HIFCR_CTCIF6; /* clear TC flag */
43         USART2->CR3 &= ~USART_CR3_DMAT; /* disable UART DMA TX */
44         DMA1_Stream6->CR &= ~DMA_SxCR_EN;
45     }
46 }
47
48 void DMA_UART_Transmit(const char *data, uint16_t len) {
49     while (DMA1_Stream6->CR & DMA_SxCR_EN); /* wait if busy */
50     DMA1_Stream6->MOAR = (uint32_t)data;
51     DMA1_Stream6->NDTR = len;
52     USART2->CR3 |= USART_CR3_DMAT; /* enable UART DMA TX */
53     DMA1_Stream6->CR |= DMA_SxCR_EN; /* start transfer */
54 }
55

```

```

56 void USART2_DMA_Init(void) {
57     RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
58     RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
59     __NOP(); __NOP();
60     GPIOA->MODER  &= ~((3UL<<4)|(3UL<<6));
61     GPIOA->MODER  |= ((2UL<<4)|(2UL<<6));
62     GPIOA->AFR[0] &= ~((0xFUL<<8)|(0xFUL<<12));
63     GPIOA->AFR[0] |= ((7UL<<8)|(7UL<<12));
64     USART2->BRR   = (390 << 4) | 10;
65     USART2->CR3    = USART_CR3_DMAR;           /* enable UART DMA RX */
66     USART2->CR1     = USART_CR1_UE | USART_CR1_TE | USART_CR1_RE;
67 }
68
69 int main(void) {
70     USART2_DMA_Init();
71     DMA1_Init();
72     DMA_UART_Transmit(txMsg, strlen(txMsg));    /* non-blocking TX */
73
74     uint16_t prevNDTR = RX_BUF_SIZE;
75     while (1) {
76         /* Detect new RX data by monitoring NDTR decrement */
77         uint16_t curNDTR = DMA1_Stream5->NDTR;
78         if (curNDTR != prevNDTR) {
79             /* New data in rxBuf -- process here */
80             prevNDTR = curNDTR;
81         }
82         __WFI();
83     }
84 }

```

Listing 5.1: Lab 5A: UART DMA TX + circular RX (No HAL)

5.3 Lab 5B — HAL Code

```

1  /* Lab 5B - UART2 DMA using HAL
2   * CubeMX: USART2 + DMA TX (Stream6 Ch4) + DMA RX (Stream5 Ch4)
3   */
4  #include "main.h"
5  #include <string.h>
6
7  UART_HandleTypeDef huart2;
8  DMA_HandleTypeDef hdma_usart2_tx;
9  DMA_HandleTypeDef hdma_usart2_rx;
10
11 #define RX_BUF_SIZE 64
12 uint8_t rxBuf[RX_BUF_SIZE];
13 volatile uint8_t txDone = 1;
14
15 /* Called when DMA TX transfer finishes */
16 void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart) {
17     if (huart->Instance == USART2) txDone = 1;
18 }
19
20 /* Called when DMA RX fills the full buffer */
21 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
22     if (huart->Instance == USART2) {
23         HAL_UART_Transmit_DMA(&huart2, rxBuf, RX_BUF_SIZE); /* echo */
24         HAL_UART_Receive_DMA(&huart2, rxBuf, RX_BUF_SIZE);  /* re-arm */
25     }
26 }
27
28 int main(void) {
29     HAL_Init();

```

```

30  SystemClock_Config();
31  MX_DMA_Init();           /* DMA controller init (CubeMX) */
32  MX_USART2_UART_Init();   /* Links DMA handles to UART */
33
34  const char *hello = "HAL DMA UART Demo -- send 64 bytes\r\n";
35  /* Non-blocking DMA transmit */
36  while (!txDone);
37  txDone = 0;
38  HAL_UART_Transmit_DMA(&huart2, (uint8_t*)hello, strlen(hello));
39
40  /* Start background DMA receive (64 bytes per cycle) */
41  HAL_UART_Receive_DMA(&huart2, rxBuf, RX_BUF_SIZE);
42
43  while (1) {
44      /* CPU is free; callbacks handle all data movement */
45      HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
46  }
47  }

```

Listing 5.2: Lab 5B: UART DMA TX + RX using HAL

[Q] Post-Lab Questions

- Q1.** DMA1 Stream 6 is assigned to USART2 TX. From RM0390 Table 28, identify two other peripherals that share DMA1 Stream 6. What happens if two peripherals attempt to use the same stream simultaneously? How is this resolved in hardware?
- Q2.** Calculate the CPU time consumed by a 1 KB UART transfer at 115200 baud using: (a) polling, (b) interrupt per byte, (c) DMA with one TC interrupt. Show working. What is the CPU utilisation percentage for each approach at $f_{\text{CPU}} = 45 \text{ MHz}$?
- Q3.** The RX DMA is circular. If the main loop stalls for 200 ms and data arrives continuously at 115200 baud, how many bytes accumulate? When does overwrite occur? What hardware or software mechanism prevents data loss?
- Q4.** Explain the DMA FIFO (**SxFCR**). When does enabling it improve throughput? Configure the TX stream to burst-transfer 4 words at a time. Write and annotate the **SxFCR** and **SxCR** register writes required.
- Q5.** Design a double-buffer DMA RX system: DMA alternates between two 64-byte buffers (**HAL_UART_RxHalfCpltCallback** and **HAL_UART_RxCpltCallback**). Sketch the state machine and write the complete HAL callback implementations.

A DMA stream can service only one request source at a time. If you configure the same stream for two peripherals simultaneously: Only the selected channel (CHSEL) is actually connected. The other peripheral's DMA requests are not serviced → data loss/stall.

Lab A Debugging Techniques

A.1 GNU Debugger (GDB) in STM32CubeIDE

STM32CubeIDE integrates a GDB frontend connected to OpenOCD over the ST-LINK SWD interface. Start a session with *Run* → *Debug* (F11). The IDE flashes the firmware and halts at `main()`.

- **Breakpoints** — Click the left margin of any source line. MCU halts when PC reaches that address.
- **Register view** — *Window* → *Show View* → *Registers*. Expand the SFR tree to inspect `GPIOA->MODER`, `USART2->SR`, `DMA1->>LISR` live.
- **Memory view** — *Window* → *Show View* → *Memory*. Enter `0x40020000` (GPIOA base) to see the full register block as hex.
- **Watch expressions** — In the Expressions view, add `head`, `tail`, or `&USART2->BRR`.
- **SWV printf** — Enable Serial Wire Viewer to redirect `printf()` over the SWO pin non-invasively (no UART needed).
- **Step Into / Over** — F5 / F6 navigate through code instruction by instruction or function by function.

A.2 Common Fault Table

Symptom	Likely Cause and Fix
LED never blinks	<code>RCC->AHB1ENR</code> GPIOAEN not set. MODER bits are 00 (input) not 01. Check BSRR write offset for reset.
UART garbage output	Wrong BRR value — recalculate with actual APB1 frequency. Check AF7 selection in <code>AFR[0]</code> .
UART TX hangs	<code>TXE</code> never set. Verify <code>USART_CR1_TE</code> and <code>USART_CR1_UE</code> both set. Check APB1 clock running.
ISR never fires	NVIC not enabled, or stale pending flag from a previous event. Check <code>EXTI->IMR</code> and <code>NVIC_EnableIRQ</code> .
ISR fires continuously	Pending flag not cleared inside ISR. Write 1 to <code>EXTI->PR</code> or read <code>USART->DR</code> .
DMA transfer never starts	Wrong stream/channel. <code>SxCR_EN</code> set before <code>SxPAR/SxMOAR/SxNDTR</code> written. <code>DMAT/DMAR</code> bit in <code>USART->CR3</code> not set.
DMA data corrupt	Buffer in wrong SRAM region. Check <code>SxMOAR</code> alignment matches <code>MSIZE</code> . Ensure no cache issue.

Lab B Register Quick Reference

B.1 RCC — Clock Enable

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;    // GPIOA
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOCEN;    // GPIOC
RCC->AHB1ENR |= RCC_AHB1ENR_DMA1EN;     // DMA1
RCC->APB1ENR |= RCC_APB1ENR_USART2EN;    // USART2
RCC->APB2ENR |= RCC_APB2ENR_SYSCFGEN;    // SYSCFG (for EXTI mux)
```

Listing B.1: Common RCC clock enables

B.2 GPIO — Bit-Field Template

```
GPIOX->MODER  &= ~(3UL << (N*2));        // clear MODER
GPIOX->MODER  |= (MODE << (N*2));          // 0=IN 1=OUT 2=AF 3=AN
GPIOX->OTYPER  &= ~(1UL << N);             // 0=Push-Pull 1=Open-Drain
GPIOX->OSPEEDR |= (3UL << (N*2));          // 11 = Very High Speed
GPIOX->PUPDR   &= ~(3UL << (N*2));          // 00=None 01=PU 10=PD
GPIOX->AFR[N/8] |= (AF << ((N%8)*4));      // Alternate Function 0-15
GPIOX->BSRR   = (1UL << N);                // Set HIGH (atomic)
GPIOX->BSRR   = (1UL << (N + 16));          // Set LOW (atomic)
```

Listing B.2: GPIO configuration template for pin N on port X

B.3 NVIC — CMSIS Helpers

```
NVIC_SetPriority(IRQn, priority);    // 0 = highest; check __NVIC_PRIO_BITS
NVIC_EnableIRQ(IRQn);                // unmask in NVIC
NVIC_DisableIRQ(IRQn);               // mask in NVIC
NVIC_GetPendingIRQ(IRQn);             // check pending status
NVIC_ClearPendingIRQ(IRQn);           // clear pending (use carefully)
__enable_irq(); __disable_irq();      // global interrupt enable / disable
```

Listing B.3: NVIC control via CMSIS intrinsics

B.4 Useful Macros

```
/* Read a bit-field */
#define READ_FIELD(reg, mask, pos) (((reg) & (mask)) >> (pos))

/* Modify a bit-field safely (not atomic -- disable IRQ if needed) */
#define MODIFY_REG(reg, mask, val) ((reg) = (((reg) & ~(mask)) | (val)))

/* Round up to next power-of-two alignment */
#define ALIGN_UP(x, a) (((x) + (a) - 1) & ~((a) - 1))

/* Memory barriers */
#define DATA_SYNC_BARRIER() __DSB()
#define INST_SYNC_BARRIER() __ISB()
```

Listing B.4: Utility macros for embedded C development

Lab C Assessment Rubric

Each lab session is graded out of **100 marks**, distributed as below. Instructors may adjust weights to align CO1/CO2/CO3 emphasis with the CSE2205 course assessment plan (Final Exam 60 %, Midterm 20 %, Class Test 10 %, Assignment/Presentation 10 %).

Component	Criteria	Marks
Pre-lab Viva	Student explains theory, key registers, and expected hardware behaviour before hands-on work	20
Demonstration	Code compiles, flashes, and shows correct hardware behaviour within the session	40
Code Quality	Correct register config, meaningful comments, no magic numbers, error handling present	20
Post-Lab Report	All post-lab questions answered; oscilloscope screenshots; waveform analysis	20
Total		100

Table C.1: Lab assessment rubric

Lab	Primary Topic	CO Mapping	Difficulty
1A / 1B	LED Blink — GPIO Output	CO1, CO2	★
2A / 2B	UART Polling Echo	CO1, CO2	★★
3A / 3B	External Interrupt (EXTI)	CO2	★★
4A / 4B	UART Interrupt + Ring Buffer	CO2	★★★
5A / 5B	UART DMA Background Transfer	CO2, CO3	★★★★

Table C.2: Lab CO mapping and difficulty rating